SYSC3110 Project 1

Data Structure Explanation:

Enum Classes

Time Complexity: O(1)

CardColor

Contains only the five necessary colours: RED, BLUE, YELLOW, GREEN, and WILD.
Prevents invalid colours and ensures type safety.
Using an enum ensures memory efficiency since only these constant values exist.

CardType

Contains only the required card types such as ONE–NINE, SKIP, REVERSE, DRAW_ONE, WILD, and WILD_DRAW_TWO.
Prevents invalid types and ensures consistent behaviour.
Memory efficient and easy to extend if new card types are added in the future.

Card Class

Data Structure: HashMap<CardType, Integer> Point_System
Time Complexity: O(1) retrieval

Stores the point values associated with each CardType.
Using a HashMap allows constant-time retrieval of point values.
The map is declared static final, ensuring immutability and centralized initialization.
Makes it easy to modify point values in one place if game rules change.
Fields colour and type are declared final, ensuring that card properties cannot change after creation.

Relevant Operation(s): getPoint()

PileSystem Class

Data Structures: Stack<Card> drawPile and Stack<Card> discardPile
Time Complexity: O(1) for push() and pop()

A stack is used because the game requires a “last in, first out” behavior when drawing and discarding cards.
Supports fast push() and pop() operations for handling card flow.
Collections.shuffle() is used to randomize the draw pile efficiently.

Relevant Operation(s): isEmpty(), size(), peek(), push(), pop(), Collections.shuffle()

Player Class

Data Structure: ArrayList<Card> hand
Time Complexity:
Access by index: O(1)
Append: O(1)

Allows dynamic hand sizes that can grow or shrink as cards are drawn or played.
Maintains the order of cards, which is useful for displaying the player's hand.
Provides efficient operations for adding and removing cards.

Relevant Operation(s): Collections.unmodifiableList(), size(), clear(), add(), remove()

UNOModel Class
Data Structure: ArrayList<Player> players and ArrayList<IView> views
Purpose: Holds all active players and registered views that observe model changes.
Reasoning and Efficiency:
ArrayList provides constant-time index retrieval (O(1)) for turn management.
The views list supports observer pattern updates via iteration (O(n) per update cycle).

Relevant Operations: addPlayer(), nextPlayer(), notifyViewers()

UNOController Class
Data Structure: Uses Java Event Model (ActionListener) mappings for user actions.
Purpose: Acts as a bridge between the View and Model.
Reasoning and Efficiency:
Delegates events to model methods directly for constant-time command dispatch.
No redundant data stored; ensures lightweight communication.

Status Enum
Purpose: Represents the current state of the game (e.g., WAITING_FOR_PLAY, GAME_OVER).
Data Structure Type: enum
Reasoning and Efficiency:
Simplifies state management without using complex condition maps.
Constant-time checks and comparisons (O(1)).